

AES-128 Security Evaluation Report

Problem: Security evaluation of an AES-128 encryption engine (Verilog) for Tiny Tapeout ASIC **Agents:** 3 | **Rounds:** 3 **Date:** 2026-03-31 **Design:** tt_um_tqv_peripheral_harness — RISC-V TinyQV + AES-128 peripheral

Participants

Agent	Role	Final Confidence
Agent A	Cryptographic Security Expert	9/10
Agent B	Hardware Side-Channel Analyst	9/10
Agent C	Fault Injection & Access Control Specialist	9/10

How AES-128 Works (The 30-Second Version)

AES-128 encrypts data in blocks of 16 bytes using a 128-bit secret key. It does this by running the data through **10 rounds** of transformations. Each round applies four operations:

1. **SubBytes** — Each byte is replaced using a lookup table called the **S-box**. This is the non-linear step that makes AES hard to crack mathematically. Think of it like a secret codebook: byte `0x53` always becomes `0xED`, etc.
2. **ShiftRows** — The 16 bytes are arranged in a 4x4 grid and each row is shifted sideways by a different amount. This spreads information across columns.
3. **MixColumns** — Each column of 4 bytes is mathematically mixed together (using Galois field multiplication). This ensures that changing one input byte eventually affects *every* output byte. Skipped in the final round.
4. **AddRoundKey** — The data is XORed with a **round key** derived from the original secret key. Each round uses a different round key, generated by the **key schedule** (key expansion algorithm).

Before round 1, there's an initial AddRoundKey (XOR plaintext with the original key). The final round (round 10) skips MixColumns. Total: 1 initial XOR + 10 rounds = the full AES-128 encryption.

This design implements all of that in hardware (Verilog) for an ASIC chip, using 10 S-boxes (the Canright composite-field design, which is smaller than a lookup table) split across two clock phases to save chip area.

Security Terminology — Plain English

What is a “Side-Channel Attack”?

Imagine you’re trying to figure out someone’s ATM PIN. You could try to hack the bank’s software (that’s a *direct* attack). Or you could stand behind them and watch which buttons they press — their shoulder movements “leak” the PIN even though the keypad is secure. That’s a **side-channel attack**: extracting secrets by observing *physical byproducts* of computation rather than breaking the math.

In hardware, the “physical byproducts” are things like:

- **Power consumption** — every transistor that switches draws a tiny bit of current. Different computations draw different amounts. If you measure the power draw of a chip precisely enough while it encrypts, you can figure out what data it’s processing.
- **Electromagnetic radiation** — switching transistors emit tiny radio waves. An antenna held close to the chip can pick these up, just like power analysis but wireless.
- **Timing** — if some operations take longer than others depending on the data, you can figure out the data by measuring how long things take.

SPA — Simple Power Analysis You hook up an oscilloscope to the chip’s power supply and record what happens during *one* encryption. If the chip is poorly designed, you can literally see the key by looking at the power trace — like watching someone type a password through a window.

In our design: The fixed 22-cycle timing and distinct two-phase S-box computation make the power trace very “readable.” An expert could potentially distinguish operations from a single trace.

DPA — Differential Power Analysis This is the big one. Instead of reading the key from one trace, you collect hundreds of power traces from different encryptions. Then you do statistics: “If key byte #1 is 0x2B, what would the power consumption look like at this moment? Does that match what I measured?” You try all 256 possible values for each byte and the correct guess produces a statistical spike. It’s like trying all possible passwords but using power measurements instead of login attempts.

In our design: The S-boxes have no “masking” (see below), so this attack works with about 500-800 traces. That’s maybe 10 minutes of measurement with a \$50 oscilloscope.

CPA — Correlation Power Analysis A smarter version of DPA. Instead of just comparing averages, it uses Pearson correlation (the same formula from your statistics class) to find which key hypothesis best predicts the measured power. Works with fewer traces.

What is “Masking”?

Masking is the main defense against power analysis. Here’s the intuition:

The attacker’s power analysis works because the chip computes $S\text{-box}(\text{plaintext XOR key})$ directly. The power consumption depends on the *actual value* of plaintext XOR key , which contains the key.

Masking adds randomness: instead of computing $S\text{-box}(X)$ directly, the chip picks a random number M and computes $S\text{-box}(X \text{ XOR } M)$. The power consumption now depends on $X \text{ XOR } M$, which is random garbage — it tells the attacker nothing about X . The chip then “unmasks” the result to get the correct answer.

It’s like if you were whispering a secret in a noisy room. Without masking, the eavesdropper hears the secret. With masking, they hear the secret mixed with random noise — useless.

In our design: There is zero masking. Every S-box computation directly processes key-dependent data. This is like whispering your secret in a silent room.

What is a “Fault Injection Attack”?

Imagine you could reach into a calculator and flip a digit mid-computation. The wrong answer it gives you, compared to the right answer, tells you something about the internal state. That’s **fault injection**.

In practice, attackers cause faults by:

- **Voltage glitching** — Every flip-flop in a chip has a “decision threshold” — a voltage level above which it reads a 1, and below which it reads a 0. The flip-flop samples its input at the rising edge of the clock. Normally the voltage is stable and the flip-flop reads correctly. But if you briefly *spike* or *dip* the power supply voltage for just a few nanoseconds right as the clock edge arrives, you shift where that threshold sits. A flip-flop that was about to store a 0 might now store a 1, or vice versa. In practice, an attacker uses a tool like the **ChipWhisperer** (~\$200) — a small board that sits between the chip’s power supply and the chip itself. The attacker programs it to inject a precise voltage dip at a specific clock cycle. For example: “On cycle 19 of the encryption, drop the voltage by 200mV for 5 nanoseconds.” Because our design always takes exactly 22 cycles, the attacker knows *exactly* which cycle is round 9 vs round 10 — there’s no guesswork. They run the encryption over and over, tweaking the glitch timing by tiny increments, until they hit the round counter at the right moment.
- **Clock glitching** — Similar principle, different mechanism. Instead of messing with voltage, the attacker inserts an extra clock edge or makes one clock period too short. Every flip-flop has a “setup time” — the minimum time the input data must be stable before the clock arrives. If you shorten

a clock period below this setup time, the flip-flop doesn't have time to settle and captures a random or wrong value. The attacker replaces the chip's crystal oscillator with their own clock source (an FPGA or signal generator) and programs it to insert one short pulse at the exact cycle they want to corrupt. Again, fixed 22-cycle timing tells them exactly when to strike. The effect is the same as voltage glitching — a bit flip in a register — but sometimes easier to target specific parts of the circuit because different logic paths have different timing margins.

- **Laser fault injection** — The most precise method. The attacker decapulates the chip (dissolves the plastic packaging with acid or mechanical grinding to expose the silicon die), then uses a focused infrared laser beam aimed at specific transistors. When the laser photons hit the silicon, they generate electron-hole pairs that temporarily flip the stored value in a flip-flop or SRAM cell. The attacker can literally choose *which register* to corrupt by aiming the laser at its physical location on the die. For example, they could target just the 2-bit FSM state register while leaving the round counter untouched, or vice versa. This requires expensive equipment (~\$50K+) and detailed knowledge of the chip's layout, so it's mainly used by government labs and smart card security evaluators. But for high-value targets (banking cards, DRM chips), it's a real and well-documented attack.

How a real attack plays out step-by-step (voltage glitch example): 1. The attacker connects a ChipWhisperer board to the chip's power pins 2. They write a known plaintext and key to the chip and trigger encryption 3. They record the correct ciphertext — this is their “golden reference” 4. They send the same plaintext again, but this time program the ChipWhisperer to glitch the power at cycle 19 (round 9 of AES) 5. The voltage dip corrupts the round counter — it jumps from 19 to 21, skipping the MixColumns operation in round 9 6. The chip outputs a faulty ciphertext and asserts **done** as if nothing happened 7. The attacker now has one correct ciphertext and one faulty ciphertext for the same plaintext 8. They repeat steps 4-7 with slightly different glitch parameters to get 2-3 more fault pairs 9. They feed the correct/faulty pairs into a DFA tool (freely available open-source software) 10. The tool mathematically solves for the key — typically in under a second

The entire attack takes maybe an hour of lab time once the setup is calibrated.

Round Skipping AES is secure because it runs 10 rounds. Each round adds more confusion. If an attacker can glitch the round counter so the chip only runs 9 rounds (or 8, or 7...), the output is dramatically weaker. Mathematicians have shown that comparing a correct 10-round output with a faulty 9-round output lets you calculate the key directly.

In our design: The round counter is a simple 5-bit number with no protection. A single voltage glitch can make it jump from 19 to 21, skipping an entire round.

The chip has no way to detect this happened.

DFA — Differential Fault Analysis This is the fault injection equivalent of DPA. The attacker: 1. Encrypts a block normally (gets correct ciphertext) 2. Encrypts the *same* block again, but injects a fault during round 9 or 10 3. Compares the correct and faulty ciphertexts

The *difference* between them is mathematically related to the key. For AES-128, just 2-4 such pairs are enough to recover the entire 128-bit key. It's like solving a system of equations — each fault gives you more equations, and eventually you have enough to solve for all unknowns.

In our design: The two-phase SubBytes splits the computation across two clock cycles, creating a perfect window to inject a fault that corrupts exactly half the state. The chip has no error detection, so it happily outputs the faulty result. This is the ideal setup for DFA.

What is “Key Zeroization”?

When you're done using a key, you should erase it — overwrite the key registers with zeros so they can't be read later. This is called **zeroization**. Good designs zeroize automatically when they detect tampering.

In our design: The soft reset command *does* zeroize (clear everything), but any software can trigger it at any time — even mid-encryption. There's no protection against an attacker using this destructively.

What are TMR and One-Hot Encoding?

These are ways to make hardware resistant to single-bit faults:

- **TMR (Triple Modular Redundancy):** Store every important value three times. If a glitch corrupts one copy, the other two “outvote” it. Like keeping three copies of an important document — if one gets damaged, you still have two good ones.
- **One-Hot Encoding:** Instead of representing states as binary numbers (IDLE=00, COUNTING=01, DONE=10), use one wire per state (IDLE=0001, COUNTING=0010, DONE=0100). If any single bit gets flipped, the result isn't a valid state (e.g., 0011 has two bits set — invalid!), so the chip can detect the fault immediately.

In our design: Neither is used. The state machine uses plain 2-bit binary encoding where flipping any single bit lands you in another perfectly valid state. The chip can't tell the difference between a legitimate state transition and a glitch.

Consensus: Ranked Vulnerability List

All three agents converged on the following ranking by Round 3:

Rank 1 — CRITICAL: Key Registers Readable via Bus

All 3 agents unanimously agreed this is the most severe vulnerability.

The problem in plain English: The secret encryption key is stored in registers at addresses 0x02 through 0x05. Any software running on the CPU can read these addresses and get the raw key back. The `key_lock` feature only prevents *overwriting* the key — it doesn't prevent *reading* it.

This is like putting your house key under the doormat and then putting a “Do Not Disturb” sign on top. The sign stops people from putting a *different* key there, but anyone can lift the sign and take your key.

Why this is #1: Every other vulnerability on this list requires special equipment, statistical analysis, or physical access. This one requires nothing — just four read commands on the bus. If you can talk to the chip at all, you can steal the key. It makes the entire encryption pointless.

What the code does wrong (in `AES_memory.v`):

```
// Read logic - key registers are returned directly
6'h02: data_out = key_mem[0]; // Anyone can read this!
6'h03: data_out = key_mem[1];
6'h04: data_out = key_mem[2];
6'h05: data_out = key_mem[3];
```

The fix: Return zeros when the key addresses are read:

```
6'h02: data_out = 32'b0; // Key is write-only
6'h03: data_out = 32'b0;
6'h04: data_out = 32'b0;
6'h05: data_out = 32'b0;
```

Impact: Complete key compromise. Makes all other security properties irrelevant. **Effort:** Low (~10 lines of RTL).

Rank 2 — CRITICAL: FSM and Round Counter Have No Integrity Protection

The problem in plain English: The chip's “brain” — the state machine that controls the encryption flow — is stored in just 2 bits. The round counter that tracks “which round are we on?” is a plain 5-bit number. Neither has any error checking.

Think of it like a combination lock with only 2 digits, and no alarm if someone spins the dial randomly. A tiny electrical disturbance (a “glitch”) can flip these bits and the chip has no idea anything went wrong.

What can go wrong with a single bit flip: - **COUNTING (01)** -> **IDLE (00)**: Encryption aborts silently. The chip stops mid-encryption and the output register holds whatever partial result was computed. Depending on when this happens, the partial output can leak key information. - **COUNTING (01)** -> **DONE (10)**: Encryption “finishes” early. The chip signals completion and outputs an incomplete ciphertext. An attacker who compares this with a correct ciphertext can mathematically recover the key (this is DFA). - **Counter 19** -> **21**: Skips one AES round entirely. The output is a 9-round AES variant that is dramatically weaker than proper 10-round AES.

Why this matters: A ChipWhisperer board (~\$200) can inject voltage glitches with nanosecond precision. Combined with the fixed 22-cycle timing (the attacker knows *exactly* when round 9 happens), targeting the counter is straightforward. With 2-4 successful glitches, the attacker recovers the full 128-bit key.

The fix: Use **one-hot encoding** for the state machine (4 flip-flops instead of 2, with illegal-state detection). Add a **shadow counter** that counts down while the main counter counts up — if they ever disagree, something went wrong. When a fault is detected, zero out all registers and refuse to output anything.

Impact: Full key recovery with physical access and ~\$200 equipment. **Effort:** Medium (~24 extra flip-flops, plus comparison logic).

Rank 3 — HIGH: Unmasked S-Boxes Enable DPA in ~500-800 Traces

The problem in plain English: The S-boxes are the heart of AES security. They’re the non-linear function that makes the cipher hard to break mathematically. But in *hardware*, the S-box computation itself can betray the key through power consumption.

Here’s why: In the very first round of AES, the S-box computes $S_box(plaintext_byte \oplus key_byte)$. The attacker knows the plaintext (they chose it). They don’t know the key. But the *power consumed* during this S-box lookup depends on the *result* — specifically, on how many bits are 1 (the “Hamming weight”).

So the attacker: 1. Sends 800 different plaintexts to the chip and measures the power each time 2. For each of the 256 possible values of key byte #1, calculates “if the key byte were this value, what would the S-box output be, and how much power should the chip draw?” 3. Checks which key guess best predicts the actual measured power 4. The best match is the real key byte 5. Repeats for all 16 key bytes independently

This is like guessing someone’s password by listening to how hard they type each key. Some keys are louder (more bits flipping = more power consumed), and the pattern reveals the password.

Aggravating factors in our design:

- **No masking:** The S-box directly processes `plaintext XOR key`. Zero randomization. The power signal is clean and directly correlated with the key.
- **Fixed timing:** Every encryption takes exactly 22 cycles. The attacker doesn’t need to “align” their traces — the S-box computation always happens at the same clock cycle. This eliminates the hardest part of DPA.
- **Two-phase split:** Upper 8 bytes are processed in one cycle, lower 8 in the next. This means the attacker gets two clean, separated windows to attack. It’s like the chip voluntarily divides its secret into two smaller problems.
- **Operand isolation backfires:** When the S-boxes are inactive, their inputs are forced to zero. When they activate, there’s a sharp jump from all-zeros to the real data. This transition is *easier* to measure than if the S-boxes were always computing something, paradoxically making the power signal cleaner for the attacker.

The fix: Implement **first-order Boolean masking** — XOR a random mask into the S-box input and correct the output. This requires a random number generator (TRNG or LFSR) and roughly doubles the S-box area.

Impact: Full key recovery with an oscilloscope (~\$50-200) and 500-800 encryption traces (~10 minutes of measurements). **Effort:** High (doubles S-box area, may not fit on the Tiny Tapeout tile).

Rank 4 — HIGH: Two-Phase SubBytes Creates Asymmetric Fault Window

The problem in plain English: To save chip area, the design processes the 16-byte SubBytes operation in two halves: the upper 8 bytes in one clock cycle (phase A), the lower 8 bytes in the next (phase B). The results from phase A are stored in a register (`sub_out_hi`), then combined with phase B’s results to form the full 16-byte output.

The security problem: there’s a one-cycle gap between the two phases. If an attacker injects a fault during this gap, they corrupt exactly 8 bytes of the SubBytes output while leaving the other 8 untouched. This creates a very specific pattern of correct-and-corrupted bytes that is *mathematically ideal* for Differential Fault Analysis.

Think of it like a factory quality check where inspectors check the left half of a product on Monday and the right half on Tuesday. If a saboteur tampers with the product on Monday night, only the right half is affected. The inspector can compare Monday’s records (good left half) with Tuesday’s results (corrupted right half) and figure out exactly what was changed. If both halves were inspected

simultaneously, the saboteur would have to tamper with everything at once, which is much harder to do precisely.

With standard DFA techniques, 2-4 such asymmetric faults are enough to recover the entire AES key.

The fix: Either process all 16 bytes in one cycle (requires 16 S-boxes — significantly more area) or add an integrity check between phases — compute a parity/checksum of the phase A output and verify it hasn't been tampered with before committing phase B.

Impact: Amplifies fault injection attacks. Reduces the number of faults needed for key recovery. **Effort:** Medium-High.

Rank 5 — HIGH: Soft Reset Race Condition

The problem in plain English: The control register has a “soft reset” bit (bit 1) that clears *everything* — key registers, plaintext, ciphertext, all internal state. Any software can trigger this at any time, including while the chip is mid-encryption.

This creates two problems:

1. **Denial of service:** Any software that can write to the control register can destroy the key at any time. No privilege check, no confirmation, no interlock. On a multi-tenant system, one misbehaving process can wipe the key for everyone.
2. **Encryption with a dead key:** If the soft reset fires while the key engine is mid-expansion, the key registers get zeroed but the key engine may have already computed some round keys from the original key. The encryption could complete using a mix of original-key-derived round keys and zero-key-derived round keys — a cryptographically broken state that could leak information about the original key.

Think of it like pulling the plug on an oven mid-bake. Some layers of the cake are cooked, others are raw. The result isn't just a bad cake — if someone analyzes the partially-cooked layers, they might be able to figure out the recipe.

The fix: Add a “busy” flag. When encryption is active, ignore soft reset commands. Only allow soft reset when the engine is idle. Also consider requiring a two-step reset process (write a specific magic value, then trigger reset) to prevent accidental or malicious resets.

Impact: Key destruction, potential encryption with weak/zero key. **Effort:** Low.

Rank 6 — MEDIUM: Fixed Execution Timing

The problem in plain English: Every single encryption takes exactly 22 clock cycles. No variation. Ever. This is normally considered a *good* thing in cryptography (constant-time execution prevents timing attacks). But in hardware, it has a downside: it makes power analysis much easier.

When the attacker collects 800 power traces, they need to “line up” the traces so that cycle 5 of trace #1 matches cycle 5 of trace #2, etc. If the timing varied randomly (say, 22-25 cycles), the attacker would need to figure out the alignment first, which is hard. With perfectly fixed timing, alignment is automatic — just start all traces from the **start** signal.

It’s like if a spy had to take photos through a window. If the curtains open at random times, the spy might miss the shot. If the curtains open at exactly 3:00 PM every day, the spy just shows up at 2:59 PM.

The fix: Insert 1-3 random dummy clock cycles (using an LFSR — a simple pseudo-random number generator) at random points during encryption. The total time varies by a few cycles, which forces the attacker to solve a trace alignment problem before they can do DPA. This alone increases the required traces by 5-10x.

Impact: Makes DPA easier by eliminating alignment challenges. **Effort:** Low.

Rank 7 — MEDIUM: No Fault Detection or Error Signaling

The problem in plain English: The chip has absolutely no way to tell you something went wrong. The **done** signal fires whether the encryption completed correctly or was completely corrupted by a fault. The **data_ready** signal is hardwired to 1 — it never indicates an error.

For a DFA attacker, this is a gift. They need both *correct* and *faulty* ciphertexts to recover the key. If the chip had fault detection, it could refuse to output the faulty result (or output all zeros), denying the attacker the data they need. Without it, every fault injection attempt gives the attacker usable data.

Think of it like a bank that processes every transaction without checking for fraud. A scammer can keep trying small fraudulent transactions, and the bank never flags any of them. Eventually the scammer has enough data to clean out the account.

The fix: Add a **fault_detected** output pin. Set it when: - The round counter goes out of expected range - The FSM enters an illegal state - A redundant computation doesn’t match

When **fault_detected** is high, output all zeros instead of the ciphertext and require a full reset before the engine works again.

Impact: Enables silent exploitation of fault injection. **Effort:** Low-Medium.

Rank 8 — LOW: No FIPS 197 Known-Answer Self-Test

The problem in plain English: The chip has no way to verify that its AES implementation is actually correct. If a manufacturing defect subtly changes how the S-box works, or if a wire is misconnected during fabrication, the chip will happily encrypt data using a broken algorithm — and nobody will know until someone tries to decrypt the output and gets garbage.

The AES specification (FIPS 197) includes test vectors: “If you encrypt THIS plaintext with THIS key, you MUST get THIS ciphertext.” A self-test runs one of these test vectors at power-on and checks the result. If it doesn’t match, the chip refuses to operate.

The fix: Store one test vector (plaintext + key + expected ciphertext = 384 bits) in ROM. On power-up, run the test. If the output doesn’t match the expected ciphertext, set an error flag and disable the engine.

Impact: Manufacturing or design defects could produce weak ciphertext undetected. **Effort:** Low.

Key Disagreements

The only substantive disagreement was **priority ordering of mitigations**:

- **Agent B** placed S-box masking at #2 (most exploitable with minimal equipment)
- **Agent C** placed FSM hardening at #1 (fault injection enables complete key recovery)
- **Agent A** placed key read protection at #1 (zero-skill instant break)

Resolution: All agents ultimately agreed that **key readability is the #1 priority** because it requires no equipment, no skill, and no multiple attempts. An attacker who can read the key directly never needs DPA or DFA. The remaining ordering follows an attacker cost/skill gradient: fault injection (#2) requires ~\$200 equipment, DPA (#3) requires ~\$50-200 + statistical processing, architectural issues (#4-5) amplify the above attacks.

Recommended Action

Immediate (before tapeout): 1. Make key registers write-only (Rank 1) — this is a ~10-line RTL change 2. Add busy interlock on soft reset (Rank 5) — prevents mid-encryption key destruction

Short-term (next design revision): 3. Harden FSM with one-hot encoding + illegal state detector (Rank 2) 4. Add parity or shadow counter to round counter (Rank 2) 5. Add FIPS 197 self-test (Rank 8) 6. Insert random dummy cycles via LFSR (Rank 6) 7. Add `fault_detected` output (Rank 7)

Long-term (if security-critical deployment intended): 8. Implement first-order Boolean masking on S-boxes (Rank 3) 9. Collapse SubBytes to single-cycle or add integrity check (Rank 4)

Unresolved Risks

1. **Area budget:** First-order masking approximately doubles S-box area. With 10 S-boxes already consuming significant tile space, masked S-boxes may not fit on a 4x2 Tiny Tapeout tile. This is a fundamental constraint that may require accepting DPA risk on this platform.
 2. **TRNG availability:** S-box masking and timing jitter both require a random number source. Tiny Tapeout designs typically lack a TRNG. An LFSR is a partial substitute but is predictable if the seed is known.
 3. **Bus privilege separation:** The current peripheral interface has no privilege signal. Implementing key read protection and reset interlocks without a privilege model means these controls are enforced by the peripheral itself, not by the system architecture. A compromised CPU can still bypass them.
 4. **Encryption-only design:** This implementation only supports encryption, not decryption. This limits its use to CTR/OFB/CFB modes. ECB mode (which this interface naturally encourages with its block-at-a-time API) is inherently insecure for multi-block messages regardless of implementation quality.
-

Debate Highlights

Most significant mind-change: Agent A (Crypto Expert) initially focused on algorithmic correctness (RotWord endianness, SubBytes ordering) but by Round 2 conceded that access control and physical attacks are more immediately weaponizable. Quote: *“An attacker will always take the easiest path, and reading the key register directly over the bus is easier than any DPA or DFA campaign.”*

Strongest cross-agent insight: Agent C identified that the two-phase SubBytes is not just a correctness issue (Agent A’s framing) or a side-channel amplifier (Agent B’s framing) but a *structural fault injection vulnerability* — it creates an asymmetric 4-byte corruption window that is mathematically ideal

for DFA. All three agents ultimately agreed this dual-threat nature elevates it above a simple design choice.

Key synthesis: Agent B’s observation that operand isolation AND-gating *paradoxically helps* attackers by sharpening the power transition was initially counterintuitive but accepted by Round 3. The lesson: defensive measures designed for one threat model (reducing idle switching) can worsen another (creating clean attack markers for DPA).